Multi-output time-series Forecasting

Air Pollution prediction using RNN

UCI ML Air Quality Dataset

1

Time-series datasets

- https://www.kaggle.com/datasets/nishantbhadauria/datasetucimlairquality
- https://www.kaggle.com/datasets/parthd144/time-series-weather-dataset
- Monash Forecasting Repository: https://forecastingdata.org/
- UCI ML Air Quality Dataset: https://www.kaggle.com/datasets/nishantbhadauria/datasetucimlairquality

2

UCI ML Air Quality Dataset

- The dataset contains 9358 instances of hourly averaged responses from an array of 5 metal oxide chemical sensors embedded in an Air Quality Chemical Multi-sensor Device.
- The device was located on the field in a significantly polluted area, at road level, within an Italian city.
- Data were recorded from March 2004 to February 2005 (one year) representing the longest freely available recordings of on field deployed air quality chemical sensor devices responses.
- Missing values are tagged with -200 value.

3

Air Quality Dataset

Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
10-03-2004	18:00:00	2.6	1360	150	11.9	1046	166	1056	113	1692	1268	13.6	48.9	0.7578
10-03-2004	19:00:00	2	1292	112	9.4	955	103	1174	92	1559	972	13.3	47.7	0.7255
10-03-2004	20:00:00	2.2	1402	88	9.0	939	131	1140	114	1555	1074	11.9	54.0	0.7502
10-03-2004	21:00:00	2.2	1376	80	9.2	948	172	1092	122	1584	1203	11.0	60.0	0.7867
10-03-2004	22:00:00	1.6	1272	51	6.5	836	131	1205	116	1490	1110	11.2	59.6	0.7888
10-03-2004	23:00:00	1.2	1197	38	4.7	750	89	1337	96	1393	949	11.2	59.2	0.7848
11-03-2004	00:00:00	1.2	1185	31	3.6	690	62	1462	77	1333	733	11.3	56.8	0.7603
11-03-2004	01:00:00	1	1136	31	3.3	672	62	1453	76	1333	730	10.7	60.0	0.7702
11-03-2004	02:00:00	0.9	1094	24	2.3	609	45	1579	60	1276	620	10.7	59.7	0.7648
11-03-2004	03:00:00	0.6	1010	19	1.7	561	-200	1705	-200	1235	501	10.3	60.2	0.7517
11-03-2004	04:00:00	-200	1011	14	1.3	527	21	1818	34	1197	445	10.1	60.5	0.7465
11-03-2004	05:00:00	0.7	1066	8	1.1	512	16	1918	28	1182	422	11.0	56.2	0.7366
11-03-2004	06:00:00	0.7	1052	16	1.6	553	34	1738	48	1221	472	10.5	58.1	0.7353
11-03-2004	07:00:00	1.1	1144	29	3.2	667	98	1490	82	1339	730	10.2	59.6	0.7417
11-03-2004	08:00:00	2	1333	64	8.0	900	174	1136	112	1517	1102	10.8	57.4	0.7408
11-03-2004	09:00:00	2.2	1351	87	9.5	960	129	1079	101	1583	1028	10.5	60.6	0.7691
11-03-2004	10:00:00	1.7	1233	77	6.3	827	112	1218	98	1446	860	10.8	58.4	0.7552
11-03-2004	11:00:00	1.5	1179	43	5.0	762	95	1328	92	1362	671	10.5	57.9	0.7352
11-03-2004	12:00:00	1.6	1236	61	5.2	774	104	1301	95	1401	664	9.5	66.8	0.7951
11-03-2004	13:00:00	1.9	1286	63	7.3	869	146	1162	112	1537	799	8.3	76.4	0.8393
11-03-2004	14:00:00	2.9	1371	164	11.5	1034	207	983	128	1730	1037	8.0	81.1	0.8736
11-03-2004	15:00:00	2.2	1310	79	8.8	933	184	1082	126	1647	946	8.3	79.8	0.8778
11-03-2004	16:00:00	2.2	1292	95	8.3	912	193	1103	131	1591	957	9.7	71.2	0.8569
11-03-2004	17:00:00	2.9	1383	150	11.2	1020	243	1008	135	1719	1104	9.8	67.6	0.8185
11-03-2004	18:00:00	4.8	1581	307	20.8	1319	281	799	151	2083	1409	10.3	64.2	0.8065
11-03-2004	19:00:00	6.9	1776	461	27.4	1488	383	702	172	2333	1704	9.7	69.3	0.8319
11-03-2004	20:00:00	6.1	1640	401	24.0	1404	351	743	165	2191	1654	9.6	67.8	0.8133
11-03-2004	21:00:00	3.9	1313	197	12.8	1076	240	957	136	1707	1285	9.1	64.0	0.7419
11-03-2004	22:00:00	1.5	965	61	4.7	749	94	1325	85	1333	821	8.2	63.4	0.6905
11-03-2004	23:00:00	1	913	26	2.6	629	47	1565	53	1252	552	8.2	60.8	0.6657
12-03-2004	00:00:00	1.7	1080	55	5.9	805	122	1254	97	1375	816	8.3	58.5	0.6438

4

Set up library

```
import numpy as np

import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
```

✓ 2.5s

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

✓ 0.5s

```
Using device: cuda
```

5

Read the Dataset

```
import pandas as pd
df = pd.read_excel("AirQualityUCI.xlsx")
```

✓ 0.9s Python

```
df.shape
```

✓ 0.0s Python

```
(9357, 15)
```

```
df.head()
```

✓ 0.0s Python

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08
0	2004-03-10	18:00:00	2.6	1360.00	150	11.881723	1045.50	166.0	1056.25	113.0	1692.00	
1	2004-03-10	19:00:00	2.0	1292.25	112	9.397165	954.75	103.0	1173.75	92.0	1558.75	
2	2004-03-10	20:00:00	2.2	1402.00	88	8.997817	939.25	131.0	1140.00	114.0	1554.50	
3	2004-03-10	21:00:00	2.2	1375.50	80	9.228796	948.25	172.0	1092.00	122.0	1583.75	
4	2004-03-10	22:00:00	1.6	1272.25	51	6.518224	835.50	131.0	1205.00	116.0	1490.00	

6

Sort the dataset according to date-time

```
# Date and Time series are strings.
#Combine Date and Time into a single datetime column
# and sort the DataFrame by this new column
df["Datetime"] = pd.to_datetime(df["Date"].astype(str) +
" " + df["Time"].astype(str),errors="coerce")
df = df.sort_values("Datetime").reset_index(drop=True)
#Once time-series data is sorted
# Keep only numeric columns for multivariate forecasting
numeric_df =df.iloc[:, 2:-1]
# Select columns from index 2 to 14 (inclusive)
```

7

Perform data cleaning

- Missing values are tagged with -200 value.

```
df.isnull().sum()
```
✓ 0.0s

```
Date             0
Time             0
CO(GT)           0
PT08.S1(CO)      0
NMHC(GT)         0
C6H6(GT)         0
PT08.S2(NMHC)    0
NOx(GT)          0
PT08.S3(NOx)     0
NO2(GT)          0
PT08.S4(NO2)     0
PT08.S5(O3)      0
T                0
RH               0
AH               0
Datetime         0
dtype: int64
```

```
# Replace dataset missing-value marker with NaN
numeric_df = numeric_df.replace(-200, np.nan)
```
✓ 0.0s

```
numeric_df.isnull().sum()
```
✓ 0.0s

```
CO(GT)           1683
PT08.S1(CO)       366
NMHC(GT)         8443
C6H6(GT)          366
PT08.S2(NMHC)     366
NOx(GT)          1639
PT08.S3(NOx)      366
NO2(GT)          1642
PT08.S4(NO2)      366
PT08.S5(O3)       366
T                 366
RH                366
AH                366
dtype: int64
```

8

Interpolate missing data

```
# Fill missing values
numeric_df = numeric_df.interpolate(method="linear", limit_direction="both")
numeric_df = numeric_df.ffill().bfill()
```

✓ 0.0s

```
numeric_df.isnull().sum()
```

✓ 0.0s

```
CO(GT)           0
PT08.S1(CO)      0
NMHC(GT)         0
C6H6(GT)         0
PT08.S2(NMHC)    0
NOx(GT)          0
PT08.S3(NOx)     0
NO2(GT)          0
PT08.S4(NO2)     0
PT08.S5(O3)      0
T                0
RH               0
AH               0
dtype: int64
```

Fill missing values using linear interpolation.

- Look at values before and after a gap
- Fills in-between using a straight line

Example:

[10, NaN, 30] → [10, 20, 30]

limit_direction="both":

- Fill gaps in the middle AND at edges (start/end of data)
- Handle any remaining NaNs:

ffill() (forward fill)

→ fills using previous value

[10, NaN] → [10, 10]

bfill() (backward fill)

→ fills using next value

[NaN, 20] → [20, 20]

9

Convert df to numpy

```
feature_names = numeric_df.columns.tolist()
#extract feature names from df for plots
num_features = data.shape[1]

print("Features used:", feature_names)

Features used: ['CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)',
'C6H6(GT)', 'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)',
'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH',
'AH']
```

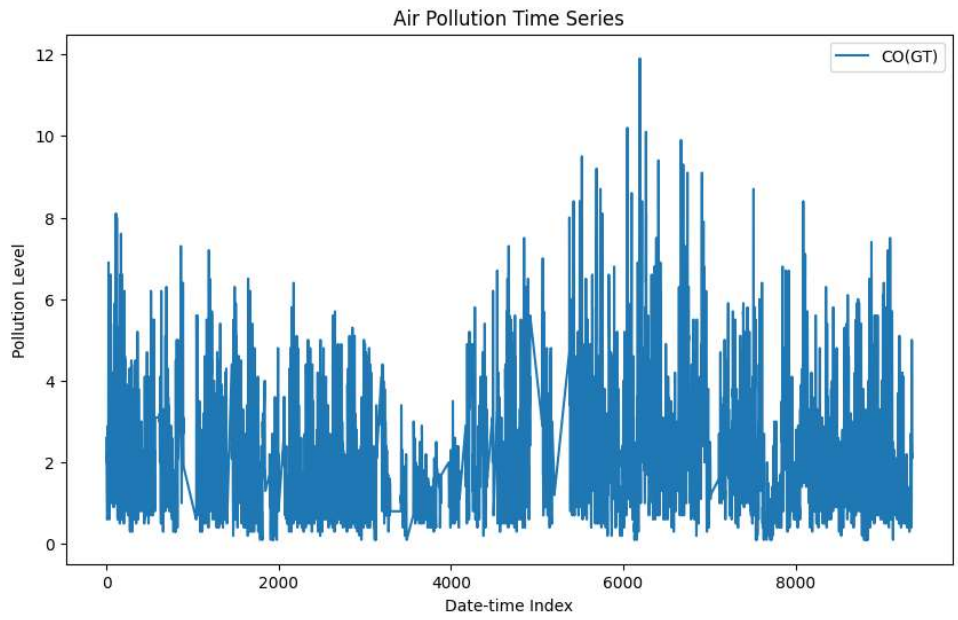
10

visualize your time series

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.plot(numeric_df.index, numeric_df[feature_names[0]], label=feature_names[0])
# Plot the first feature as an example
plt.title('Air Pollution Time Series')
plt.xlabel('Date-time Index')
plt.ylabel('Pollution Level')
plt.legend()
plt.show()
```

11

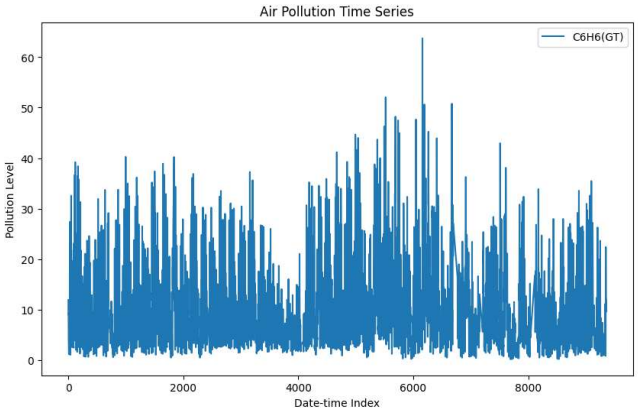
Time index v/s CO(GT)



12

Another feature

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.plot(numeric_df.index,
numeric_df[feature_names[3]],
         label=feature_names[3])
# Plot the fourth feature as an example
plt.title('Air Pollution Time Series')
plt.xlabel('Date-time Index')
plt.ylabel('Pollution Level')
plt.legend()
plt.show()
```



13

Convert df to numpy

```
data = numeric_df.values.astype(np.float32)
print("Data shape:", data.shape)
Data shape: (9357, 13)
```

```
data
✓ 0.0s
array([[2.5999999e+00, 1.3600000e+03, 1.5000000e+02, ..., 1.3600000e+01,
        4.8875000e+01, 7.5775385e-01],
       [2.0000000e+00, 1.2922500e+03, 1.1200000e+02, ..., 1.3300000e+01,
        4.7700001e+01, 7.2548747e-01],
       [2.2000000e+00, 1.4020000e+03, 8.8000000e+01, ..., 1.1900000e+01,
        5.3975002e+01, 7.5023907e-01],
       ...,
       [2.4000001e+00, 1.1420000e+03, 2.7500000e+02, ..., 2.6900000e+01,
        1.8350000e+01, 6.4064878e-01],
       [2.0999999e+00, 1.0025000e+03, 2.7500000e+02, ..., 2.8324999e+01,
        1.3550000e+01, 5.1386589e-01],
       [2.2000000e+00, 1.0707500e+03, 2.7500000e+02, ..., 2.8500000e+01,
        1.3125000e+01, 5.0280368e-01]], shape=(9357, 13), dtype=float32)
```

14

Train/Test Split

```
INPUT_STEPS = 10
OUTPUT_STEPS = 5
TRAIN_RATIO = 0.8
BATCH_SIZE = 4
```

✓ 0.0s

then normalize

```
#Train/Test split: 80:20,
# with train further split into train & validation sets
split_idx = int(len(data) * TRAIN_RATIO)
train_data1 = data[:split_idx]
test_data = data[split_idx:]
#split train into train and validation sets (80:20)
val_split_idx = int(len(train_data1) * 0.8)
train_data = train_data1[:val_split_idx]
val_data = train_data1[val_split_idx:]
```

✓ 0.0s

```
train_data.shape,test_data.shape,val_data.shape
```

✓ 0.0s

```
((5988, 13), (1872, 13), (1497, 13))
```

15

Data normalization

- Column wise normalization is more appropriate for time series forecasting
- as it preserves the temporal relationships between features
- while ensuring that each feature contributes equally to the model training.
- Row-wise normalization would distort the temporal dynamics
- Treating each time step independently is not ideal for sequence modeling tasks using RNNs.
- we will use column-wise normalization instead of Normalizer class & fit_transform()
- Standard Scaler can also be used as it will also normalize column wise.

```
from sklearn.preprocessing import normalize
#Experiment all 'l2'/'l1'/'max'
train_data_norm = normalize(train_data, norm='l2', axis=0)
#axis=0 for column-wise normalization
test_data_norm = normalize(test_data, norm='l2', axis=0)
val_data_norm = normalize(val_data, norm='l2', axis=0)
```

16

Time-series chunking into input-output pairs for supervised learning

```
INPUT_STEPS = 10
OUTPUT_STEPS = 5
TRAIN_RATIO = 0.8
BATCH_SIZE = 4
```

✓ 0.0s

```
#input_steps: number of past time steps used as input=10
#output_steps: number of future time steps to predict=5
import numpy as np

def create_sequences(data_array, input_steps=10, output_steps=5, overlap=1):
    X, y = [], []
    total_len = len(data_array)

    stride = input_steps - overlap
    if stride <= 0:
        raise ValueError("overlap must be smaller than input_steps")

    for i in range(0, total_len - input_steps - output_steps + 1, stride):
        X.append(data_array[i:i + input_steps])
        y.append(data_array[i + input_steps:i + input_steps + output_steps])

    return np.array(X, dtype=np.float32), np.array(y, dtype=np.float32)
```

✓ 0.0s

17

Preparing train/Test data with chunking

```
X_train, y_train = create_sequences(train_data_norm, INPUT_STEPS, OUTPUT_STEPS, overlap=8)
X_test, y_test = create_sequences(test_data_norm, INPUT_STEPS, OUTPUT_STEPS, overlap=8)
X_val, y_val = create_sequences(val_data_norm, INPUT_STEPS, OUTPUT_STEPS, overlap=8)

print("X_train:", X_train.shape, "y_train:", y_train.shape)
print("X_test :", X_test.shape,  "y_test :", y_test.shape)
print("X_val :", X_val.shape,  "y_val :", y_val.shape)
```

✓ 0.0s

```
X_train: (2987, 10, 13) y_train: (2987, 5, 13)
X_test : (929, 10, 13) y_test : (929, 5, 13)
X_val : (742, 10, 13) y_val : (742, 5, 13)
```

18

Create a custom dataset by extending **PyTorch's** Dataset class

```
class TimeSeriesDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

✓ 0.0s

`self.X = torch.tensor(X, dtype=torch.float32)`
Converts X into a PyTorch tensor (float type).
→ Needed because models work with tensors, not NumPy/lists.

def __init__(self, X, y):

Runs when you create the dataset. Takes:

X → inputs (features / sequences)

y → targets (labels/sequences)

return self.X[idx], self.y[idx]

Returns:

one input sequence

its corresponding target sequence

→ This is what each batch is built from.

19

Defining Tensor dataset and DataLoader

```
train_dataset = TimeSeriesDataset(X_train, y_train)
test_dataset = TimeSeriesDataset(X_test, y_test)
val_dataset = TimeSeriesDataset(X_val, y_val)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
```

✓ 0.0s

- Each data point will be arranged in shape **(batch, seq_len, features)**

20

Define model

```
class LSTMForecaster(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(13, 10, batch_first=True)
        self.fc = nn.Linear(10, 5 * 13)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = self.fc(out[:, -1, :])
        return out.view(-1, 5, 13)
```

21

RNN model forward pass

```
self.lstm = nn.LSTM(input_size=13, hidden_size=10,
num_layers=1, batch_first=True)
```

✓input_size=13 → each timestep has 13 features

✓hidden_size=10 → LSTM outputs 10 features per timestep

✓num_layers=1 → single LSTM layer

✓batch_first=True → input shape is (batch, seq_len, features)

```
self.fc = nn.Linear(10, 5*13)
```

• Takes the 10 features from LSTM → outputs 5 values with 13 features (usually predicting next 5 timesteps)

22

RNN model forward pass

```
out = self.fc(out[:, -1, :])
```

✓out[:, -1, :] → take last timestep output only

```
return out.view(-1, 5, 13)
```

✓from LSTM layer which has 10 neurons/features → shape (batch, 10)

✓from (batch_size, seq_len, hidden_size)

✓shape: (batch, 10) & pass it through fc → (batch, 5)

```
out, _ = self.lstm(x)
```

✓Pass input through LSTM (batch, seq_len=10, 13)

✓out shape: (batch, seq_len=5, 13)

✓_ is hidden state (ignored)

23

Instantiating model with a scheduler

```
model = LSTMForecaster().to(device)
lr=0.0001
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
criterion = nn.MSELoss()
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=10)
```

✓The scheduler reduces learning rate when your model stops improving.

✓Scheduler directly updates the Adam optimizer with `optimizer.param_groups[i]['lr']`

```
patience=10
```

✓Number of epochs to **wait without improvement**

✓After 10 "bad" epochs → reduce learning rate

✓ "Bad" = monitored metric (usually validation loss) not improving

24

Method for Training 1 epoch

```python
def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    running_loss = 0.0

    for X_batch, y_batch in loader:
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()
        preds = model(X_batch)
        loss = criterion(preds, y_batch)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * X_batch.size(0)

    return running_loss / len(loader.dataset)
```

✓ 0.0s

25

Method for evaluation

```python
def evaluate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    all_preds = []
    all_targets = []

    with torch.no_grad():
        for X_batch, y_batch in loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            preds = model(X_batch)
            loss = criterion(preds, y_batch)

            running_loss += loss.item() * X_batch.size(0)
            all_preds.append(preds.cpu())
            all_targets.append(y_batch.cpu())

    avg_loss = running_loss / len(loader.dataset)
    all_preds = torch.cat(all_preds, dim=0).numpy()
    all_targets = torch.cat(all_targets, dim=0).numpy()

    return avg_loss, all_preds, all_targets
```

✓ 0.0s

26

One simple Training

```
EPOCHS = 30
train_losses = []
val_losses = []

for epoch in range(EPOCHS):
    train_loss = train_one_epoch(model, train_loader, optimizer, criterion, device)
    val_loss, _, _ = evaluate(model, val_loader, criterion, device)

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    print(f"Epoch [{epoch+1}/{EPOCHS}] | Train Loss: {train_loss:.6f} | Val Loss: {val_loss:.6f}")
```

✓ 30.2s

27

Simple Training

```
Epoch [1/30] | Train Loss: 0.019268 | Val Loss: 0.002981
Epoch [2/30] | Train Loss: 0.000511 | Val Loss: 0.000288
Epoch [3/30] | Train Loss: 0.000026 | Val Loss: 0.000282
Epoch [4/30] | Train Loss: 0.000026 | Val Loss: 0.000269
Epoch [5/30] | Train Loss: 0.000025 | Val Loss: 0.000257
Epoch [6/30] | Train Loss: 0.000025 | Val Loss: 0.000244
Epoch [7/30] | Train Loss: 0.000024 | Val Loss: 0.000249
Epoch [8/30] | Train Loss: 0.000024 | Val Loss: 0.000241
Epoch [9/30] | Train Loss: 0.000024 | Val Loss: 0.000228
Epoch [10/30] | Train Loss: 0.000024 | Val Loss: 0.000227
Epoch [11/30] | Train Loss: 0.000024 | Val Loss: 0.000223
Epoch [12/30] | Train Loss: 0.000024 | Val Loss: 0.000223
Epoch [13/30] | Train Loss: 0.000023 | Val Loss: 0.000226
Epoch [14/30] | Train Loss: 0.000023 | Val Loss: 0.000218
Epoch [15/30] | Train Loss: 0.000023 | Val Loss: 0.000217
Epoch [16/30] | Train Loss: 0.000023 | Val Loss: 0.000218
Epoch [17/30] | Train Loss: 0.000023 | Val Loss: 0.000218
Epoch [18/30] | Train Loss: 0.000023 | Val Loss: 0.000214
Epoch [19/30] | Train Loss: 0.000023 | Val Loss: 0.000210
Epoch [20/30] | Train Loss: 0.000023 | Val Loss: 0.000211
Epoch [21/30] | Train Loss: 0.000023 | Val Loss: 0.000211
Epoch [22/30] | Train Loss: 0.000023 | Val Loss: 0.000209
Epoch [23/30] | Train Loss: 0.000023 | Val Loss: 0.000206
Epoch [24/30] | Train Loss: 0.000023 | Val Loss: 0.000213
Epoch [25/30] | Train Loss: 0.000023 | Val Loss: 0.000207
...
Epoch [27/30] | Train Loss: 0.000022 | Val Loss: 0.000200
Epoch [28/30] | Train Loss: 0.000022 | Val Loss: 0.000198
Epoch [29/30] | Train Loss: 0.000022 | Val Loss: 0.000213
Epoch [30/30] | Train Loss: 0.000022 | Val Loss: 0.000198
```

28

Training with Early Stopping

Hold-out validation

29

Training with Early Stopping

```python
#Training with early stopping based on validation loss
EPOCHS = 100
PATIENCE = 5   # stop if no improvement for 5 epochs
train_losses = []
val_losses = []
best_val_loss = float('inf')
epochs_no_improve = 0

for epoch in range(EPOCHS):
    train_loss = train_one_epoch(model, train_loader, optimizer, criterion, device)
    val_loss, _, _ = evaluate(model, val_loader, criterion, device)

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    print(f"Epoch [{epoch+1}/{EPOCHS}] | Train Loss: {train_loss:.6f} | Val Loss: {val_loss:.6f}")

    # Early stopping logic
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        epochs_no_improve = 0

        # optional: save best model
        torch.save(model.state_dict(), "best_model.pt")
    else:
        epochs_no_improve += 1

    if epochs_no_improve >= PATIENCE:
        print(f"Early stopping triggered at epoch {epoch+1}")
        break
```

✓ 29.1s

30

Early stopping

```
Epoch [1/100] | Train Loss: 0.021260 | Val Loss: 0.006792
Epoch [2/100] | Train Loss: 0.001837 | Val Loss: 0.000336
Epoch [3/100] | Train Loss: 0.000036 | Val Loss: 0.000261
Epoch [4/100] | Train Loss: 0.000026 | Val Loss: 0.000259
Epoch [5/100] | Train Loss: 0.000025 | Val Loss: 0.000256
Epoch [6/100] | Train Loss: 0.000025 | Val Loss: 0.000249
Epoch [7/100] | Train Loss: 0.000025 | Val Loss: 0.000238
Epoch [8/100] | Train Loss: 0.000025 | Val Loss: 0.000246
Epoch [9/100] | Train Loss: 0.000024 | Val Loss: 0.000229
Epoch [10/100] | Train Loss: 0.000024 | Val Loss: 0.000237
Epoch [11/100] | Train Loss: 0.000024 | Val Loss: 0.000234
Epoch [12/100] | Train Loss: 0.000024 | Val Loss: 0.000227
Epoch [13/100] | Train Loss: 0.000024 | Val Loss: 0.000222
Epoch [14/100] | Train Loss: 0.000024 | Val Loss: 0.000218
Epoch [15/100] | Train Loss: 0.000024 | Val Loss: 0.000231
Epoch [16/100] | Train Loss: 0.000024 | Val Loss: 0.000215
Epoch [17/100] | Train Loss: 0.000024 | Val Loss: 0.000229
Epoch [18/100] | Train Loss: 0.000024 | Val Loss: 0.000224
Epoch [19/100] | Train Loss: 0.000024 | Val Loss: 0.000212
Epoch [20/100] | Train Loss: 0.000023 | Val Loss: 0.000208
Epoch [21/100] | Train Loss: 0.000023 | Val Loss: 0.000215
Epoch [22/100] | Train Loss: 0.000023 | Val Loss: 0.000215
Epoch [23/100] | Train Loss: 0.000023 | Val Loss: 0.000210
Epoch [24/100] | Train Loss: 0.000023 | Val Loss: 0.000201
Epoch [25/100] | Train Loss: 0.000022 | Val Loss: 0.000207
...
Epoch [27/100] | Train Loss: 0.000022 | Val Loss: 0.000209
Epoch [28/100] | Train Loss: 0.000022 | Val Loss: 0.000205
Epoch [29/100] | Train Loss: 0.000021 | Val Loss: 0.000203
Early stopping triggered at epoch 29
```

31

Test dataset evaluation

```
# evaluation of model on a separate Test dataset
test_loss, predictions, targets = evaluate(model, test_loader, criterion, device)
print("\nFinal Test Loss:", test_loss)
```

✓ 0.1s

```
Final Test Loss: 0.00015244884945593063
```

```
predictions.shape, targets.shape
```

✓ 0.0s

```
((929, 5, 13), (929, 5, 13))
```

```
predictions
```

✓ 0.0s

```
array([[[0.013459  , 0.01268571, 0.01055978, ..., 0.01326722,
         0.01091979, 0.01791361],
        [0.00899873, 0.00950988, 0.01102467, ..., 0.01445171,
         0.01629825, 0.01068425],
        [0.01150866, 0.00991753, 0.0159183 , ..., 0.01176178,
         0.01613829, 0.01335473],
        [0.00643498, 0.01167336, 0.01287162, ..., 0.01705472,
         0.01290327, 0.01789896],
        [0.01147472, 0.01455078, 0.01295697, ..., 0.01104153,
         0.01132893, 0.0107827 ]],

       [[0.01247405, 0.01212503, 0.01050295, ..., 0.01285351,
         0.01067092, 0.01805273],
        [0.00724448, 0.00955001, 0.01062577, ..., 0.01455101,
         0.01685923, 0.01146751],
        [0.01043497, 0.0100032 , 0.01533817, ..., 0.01202442,
         0.01562318, 0.01346351],
```

32

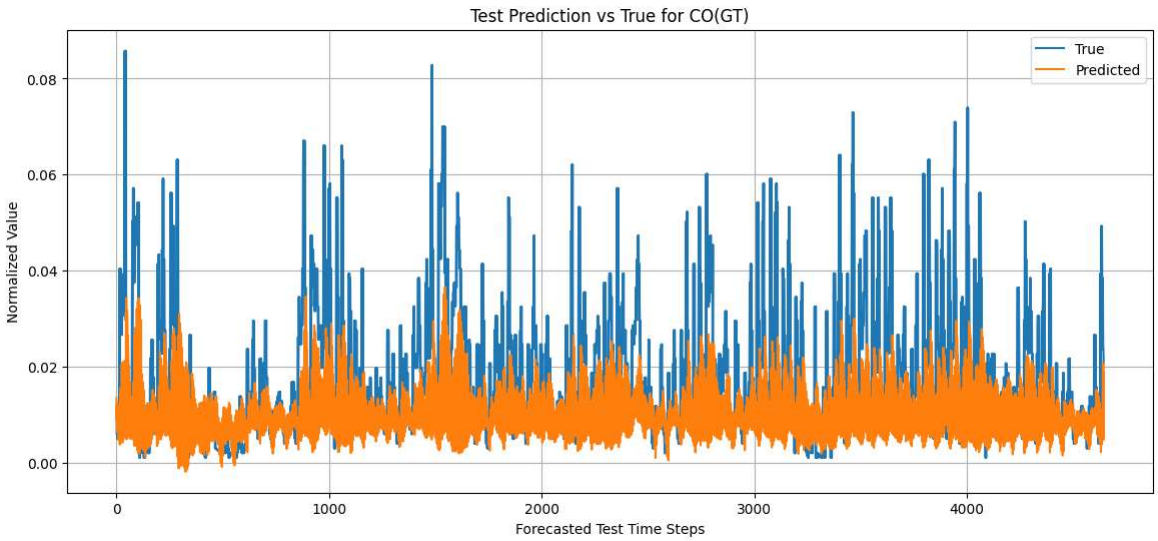
Plot predictions v/s True labels on test dataset

```
# Plot prediction vs true values
PLOT_FEATURE="CO(GT)"  # Change this to next feature
if PLOT_FEATURE not in feature_names:
    raise ValueError(f"{PLOT_FEATURE} not found. Choose from: {feature_names}")
feature_idx = feature_names.index(PLOT_FEATURE)
pred_feature = predictions[:, :, feature_idx].reshape(-1)
true_feature = targets[:, :, feature_idx].reshape(-1)
plt.figure(figsize=(14, 6))
plt.plot(true_feature, label="True")
plt.plot(pred_feature, label="Predicted")
plt.title(f"Test Prediction vs True for {PLOT_FEATURE}")
plt.xlabel("Forecasted Test Time Steps")
plt.ylabel("Normalized Value")
plt.legend()
plt.grid(True)
plt.show()
```

✓ 0.0s

33

Plot predictions v/s True labels



34